TECH STACK DOCUMENT

1. Introduction

This document describes the complete technology stack, architecture, and implementation logic for the project. The objective is to build a scalable, secure, maintainable, and high-performance application capable of supporting thousands of users.

---

2. Frontend Technology Stack

2.1 React.js

Purpose

React.js is used to build the user interface.

Why React?

- Component-based architecture
- Reusable UI components
- Fast rendering using Virtual DOM
- Large ecosystem support

Logic

Each page is divided into reusable components such as:

- Navbar
- Sidebar
- Dashboard Cards
- Tables
- Forms
- Charts

Components communicate through:

- Props
- State Management
- API Calls

2.2 Next.js

Purpose

Provides:

- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- Better SEO
- Faster Loading

Benefits

- Improved performance
- Search engine optimization
- Better routing system

2.3 TypeScript

Purpose

Adds static typing to JavaScript.

Benefits

- Reduces bugs
- Better IDE support
- Easier maintenance

Example:

User Interface:

- User ID
- Name
- Email
- Role

TypeScript ensures correct data structure.

2.4 Tailwind CSS

Purpose

UI Styling Framework

Features

- Utility-first CSS
- Fast development
- Responsive design
- Consistent UI

Benefits

- Smaller CSS files
- Better maintainability
- Modern appearance

2.5 Redux Toolkit

Purpose

Global State Management

Stores Data Such As:

- Logged-in user
- Authentication token
- Application settings
- Notifications

Benefits

- Predictable state
- Easy debugging
- Centralized management

---

3. Backend Technology Stack

3.1 Node.js

Purpose

Backend Runtime Environment

Advantages

- High performance
- Non-blocking architecture
- Event-driven system

Suitable For

- APIs
- Real-time applications
- High traffic systems

3.2 Express.js

Purpose

Backend Framework

Responsibilities

- Routing
- Middleware
- Authentication
- API Management

Example Routes

GET /users

POST /login

PUT /profile

DELETE /user

3.3 Authentication System

Technology

JWT (JSON Web Token)

Flow

1. User Login
2. Credentials Verification
3. JWT Generation
4. Token Sent to Frontend
5. Token Stored
6. Protected APIs Verify Token

Benefits

- Stateless authentication
- High performance

- Secure access

3.4 Role-Based Access Control

Roles

Admin Manager User

Example Permissions

Admin:

- Create Users
- Delete Users
- Manage System

Manager:

- View Reports
- Manage Team

User:

- Access Own Data

---

4. Database Layer

4.1 PostgreSQL

Purpose

Primary Relational Database

Stores

- Users
- Transactions
- Orders
- Reports
- Settings

Benefits

- ACID Compliance
- Reliability
- Strong Security
- Complex Queries

4.2 Database Design

Tables

Users

Orders

Products

Payments

Reports

Notifications

Relationships

User ↓ Orders ↓ Payments

4.3 Database Optimization

Methods

- Indexing
- Query Optimization
- Pagination
- Connection Pooling

Benefits

- Faster Queries
- Better Performance
- Reduced Server Load

---

5. Caching Layer

5.1 Redis

Purpose

In-memory database used for:

- Caching
- Session Management
- Temporary Storage

Cached Data

- Dashboard Statistics
- Frequently Accessed Records
- API Responses

Benefits

- Faster response times
- Reduced database load

---

6. API Architecture

API Flow

Frontend ↓ API Gateway ↓ Backend Services ↓ Database

Responsibilities

Frontend:

- User Interaction

Backend:

- Business Logic

Database:

- Data Storage

Redis:

- Fast Retrieval

---

7. Cloud Infrastructure

7.1 AWS

Services

EC2 RDS S3 CloudFront IAM

AWS EC2

Purpose: Application Hosting

Benefits:

- Scalability
- Flexibility
- Cost Optimization

AWS RDS

Purpose: Managed PostgreSQL Database

Benefits:

- Automatic Backups
- High Availability
- Monitoring

AWS S3

Purpose: File Storage

Stores:

- Images
- PDFs
- Documents
- Reports

Benefits:

- Unlimited Storage
- Durability
- Security

AWS CloudFront

Purpose: Content Delivery Network

Benefits:

- Faster loading
- Global distribution

- Reduced latency

---

8. DevOps

8.1 Docker

Purpose

Containerization

Benefits

- Consistent environments
- Easy deployment
- Scalability

8.2 GitHub Actions

Purpose

CI/CD Pipeline

Process

Code Push ↓ Testing ↓ Build ↓ Deployment

Benefits

- Automated deployment
- Faster releases
- Reduced human error

---

9. Security Architecture

Authentication

JWT

OAuth 2.0

Multi-Factor Authentication

Data Security

Encryption

At Rest: AES-256

In Transit: HTTPS/TLS

API Security

Rate Limiting

Request Validation

Input Sanitization

CSRF Protection

XSS Protection

---

10. Monitoring & Logging

Tools

- Grafana
- Prometheus
- Winston Logger

Monitoring

CPU Usage

Memory Usage

Response Time

Error Rates

Server Health

---

11. System Architecture Diagram

Client ↓ React + Next.js ↓ API Gateway ↓ Node.js + Express.js ↓ Redis Cache ↓ PostgreSQL Database ↓ AWS Infrastructure

12. Scalability Strategy

Horizontal Scaling

Add More Servers

Vertical Scaling

Increase Server Resources

Database Scaling

Read Replicas

Sharding

Caching

Load Balancing

---

13. Performance Optimization

Frontend:

- Lazy Loading
- Code Splitting
- Image Optimization

Backend:

- Query Optimization
- Caching
- Compression

Database:

- Indexing
- Connection Pooling

Infrastructure:

- CDN
- Load Balancers

---

14. Conclusion

The proposed technology stack combines React.js, Next.js, Node.js, Express.js, PostgreSQL, Redis, AWS, Docker, and GitHub Actions to create a modern, scalable, secure, and enterprise-grade application.

This architecture ensures:

- High Performance
- Security
- Scalability
- Reliability
- Maintainability
- Future Growth Support